

THE FULL TRANSFORMER ARCHITECTURE

How does AI understand true meaning?

Self-Attention, Multi-Head Spaces,
Cross-Attention, and GPU Tradeoffs.

Dibayendu Mukherjee

AI Research Engineer · Systems Engineer

The Sequential Bottleneck

Before Transformers, NLP relied heavily on recurrent models (RNNs/LSTMs).

But reading word-by-word introduced two massive flaws:

- 1. Memory Degradation:** By the end of a long sentence, the network often forgot the beginning.
- 2. The Hardware Bottleneck:** Token N had to wait for Token $N - 1$ to finish processing.

The Realization

As long as we processed sequences one step at a time, we could never fully utilize the massive parallel computing power of modern GPUs.

We needed a way to process everything at once.

The Intuition: The Cocktail Party

Imagine you are at a crowded cocktail party and you overhear the word **"apple"**.

Are they talking about the fruit or the tech company?

Your brain doesn't read the conversation sequentially to figure it out. It instantly scans the surrounding words for context clues.

Context A: Technology

- "Silicon"
- "Macbook"
- "Steve Jobs"

Context B: Fruit

- "Pie"
- "Orchard"
- "Crisp"

This is exactly how **Self-Attention** works.
Instead of reading left-to-right, every single word looks at every other word in the sentence simultaneously, asking:

"Which of these words modifies my meaning?"

The Mechanism: Q, K, and V

To achieve this, the Self-Attention mechanism maps every word into three distinct vectors:

1. The Query (Q)

"What am I looking for?"

This represents the current word that is trying to gather context from the rest of the sentence.

2. The Key (K)

"What do I have?"

This represents the information or "tags" that every other word is holding. The network checks if a Key matches the Query.

3. The Value (V)

"What is my actual meaning?"

If a Query and a Key match well, the network pulls in this Value to update the original word's context.

Breaking Down the Math

It looks intimidating, but it is just a remarkably elegant way to compute relationships.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- QK^T (**Dot Product**): We multiply the Queries by the Keys. The dot product measures *similarity*. If word A is highly relevant to word B, this score explodes.
- $\sqrt{d_k}$ (**Scaling**): We divide by the square root of the dimension size to keep the gradients stable during backpropagation.
- **softmax**: This converts the raw scores into percentages (weights that sum to 1). Now we know exactly how much "attention" word A should pay to word B.
- $\times V$ (**The Update**): We multiply those percentages by the actual Values, creating a new, rich, context-aware representation of our word.
- **Spaces of Q , K , and V** : Q and K are projected into the *same vector space*, so their dot product is meaningful — it measures alignment between "what I'm looking for" (Query) and "what I contain" (Key). V lives in the *embedding space*, carrying the semantic content of the words. The attention weights (from QK^T) decide how much of each Value contributes to the final representation.

Multi-Head Attention

Why stop at a single perspective?

Instead of one attention function, Transformers project Q, K, and V into **multiple lower-dimensional subspaces**.

The Theory

We say that multiple heads allow the model to capture different relationships (syntactic vs semantic vs positional) simultaneously.

The Reality

The model is never explicitly told to split the work! It just discovers these specializations organically through gradient descent.

An Emergent Property

Separate parameter subspaces naturally increase representational capacity. We didn't hand-engineer syntactic rules into the heads; the network learned that specializing was the most efficient way to minimize the loss function.

Example: - One head might learn that *"eat"* (verb) strongly attends to *"apple"* (noun), capturing syntactic structure. - Another head might focus on semantic roles, linking *"dog"* with *"bark"*. - Yet another head could emphasize positional context, such as connecting *"yesterday"* with the verb tense. These heads are not manually assigned — the model discovers these verb-noun and semantic relationships organically.

The Decoder's Magic: Cross-Attention

While self-attention looks inward, cross-attention looks outward. It is the bridge between the Encoder and the Decoder.

Self-Attention: *"What in my own past should I pay attention to?"*

Cross-Attention: *"What in the external context should I incorporate?"*

In the Decoder, the **Queries** come from the target sequence (the words we are generating).

But the **Keys** and **Values** come directly from the Encoder's final output (the source sentence).

(Recall: the **Encoder** compresses the source input into a rich representation, while the **Decoder** expands that representation into the target output. Source: Previous AE Post.)

Every single token generated dynamically asks the entire source sentence: **"Which part of you is relevant to what I'm trying to say next?"** This creates a differentiable, dynamic alignment that powers modern translation.

Example: Suppose the source sentence (Encoder input) is: *"Le chat dort"* (French).
- The Decoder is trying to generate the English translation.
- When predicting the word *"cat"*, the Query comes from the partially generated target sequence.
- The Keys and Values come from the Encoder's representation of *"Le chat dort"*.
- Cross-attention aligns the Query with the Encoder's output, discovering that *"chat"* is the most relevant part.
- This dynamic alignment ensures the Decoder picks the correct word *"cat"* in context.

The Engineering Genius

The brilliance of the "Attention Is All You Need" paper wasn't just linguistic. It was an engineering masterclass.

The Cost

Memory complexity became $O(N^2)$. Because every word compares itself to every other word, doubling the sentence length quadruples the memory required. This is why LLMs struggle with infinite context windows.

The Reward

Complete Parallelization. There are no sequential loops. The entire computation is just massive matrix multiplication happening across thousands of GPU cores simultaneously.

The Real Breakthrough

By shifting the bottleneck from sequential processing to memory bandwidth, transformers unlocked the ability to train on terabytes of data in parallel. They traded algorithmic elegance for hardware utilization.

Example: - Imagine a sentence with 1,000 words. - Self-attention requires each word to compare itself with all 999 others, about 1,000,000 comparisons (N^2). - That's the **cost**: memory usage explodes as sequence length grows. - But the **reward**: all those comparisons can be done at once using GPU matrix multiplication. Instead of whispering word-by-word like an RNN, it's like blasting the whole sentence through thousands of speakers simultaneously.

THE ENGINEERING TAKEAWAY

Sometimes the best algorithm isn't the one that is theoretically optimal.

It is the one that best exploits the underlying hardware.

The biggest leaps in AI didn't come from forcing networks to read better. They came from restructuring the math so GPUs could compute it faster.

Building AI systems by understanding the mathematics,
engineering, and intuition behind every architecture.

GitHub: github.com/MrHeavenly
Portfolio: mrheavenly.github.io
LinkedIn: linkedin.com/in/dibayendu-mukherjee-bb897b267

Dibayendu Mukherjee